

Reviewer Pack — Algebraic Topology of Boolean Functions in Communication Com...

Entry e945a38a700b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 17:22:55 UTC

Algebraic Topology of Boolean Functions in Communication Complexity

Entry ID: e945a38a700b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 17:22:55 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology **Field B** (complexity object): Communication Complexity

Statement:

For every communication problem with deterministic protocol complexity $C(n)$, there exists an algebraic topology invariant, specifically the Euler characteristic of a certain Čech complex constructed from the input space of the Boolean functions defining the problem, such that $E[\lambda_C(\check{C})] = \Theta(C(n))$, where λ_C denotes the Euler characteristic of the Čech complex.

Rationale (proposer's reasoning):

Algebraic topology provides powerful tools for studying topological properties of spaces. If an algebraic topological invariant can be shown to capture the complexity of communication protocols, it might provide new insights into the structure of Boolean functions and their relationships in communication complexity.

Taxonomy category: AlgebraicTopologyCommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d7d7387234875edf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the average Euler characteristic $\lambda_C(\check{C})$ of the Čech complex across multiple random seeds is within $\Theta(C(n))$ of the deterministic protocol complexity $C(n)$, with a p-value from Spearman's rank correlation coefficient less than or equal to 0.05. The conjecture is falsified if the average $\lambda_C(\check{C})$ deviates significantly from $\Theta(C(n))$, with a p-value greater than 0.05.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic topology" AND "communication complexity" AND "Boolean functions" - "Euler characteristic" AND Čech complex AND communication protocol" - "deterministic protocol complexity" AND algebraic topology invariant AND Boolean function

Top relevant hits considered: - [s2:2205.14424] Topological aspects of Boolean functions - [s2:9412c6e66560861b172a6c37c94e55f956b46be3] Functional Inversion and Communication Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_communication_problem(n):
        # Generate a random communication problem with n inputs
        return [random.choice([0, 1]) for _ in range(2**n)]

    def construct_input_space(problem):
        # Construct the input space of Boolean functions for the given problem
        input_space = []
        for i in range(len(problem)):
            if problem[i] == 0:
                input_space.append((i, 0))
            else:
                input_space.append((i, 1))
        return input_space

    def compute_euler_characteristic(input_space):
        # Compute the Euler characteristic of the Čech complex
        n = len(input_space)
        if n == 0:
            return 0
```

```

elif n == 1:
    return 1
else:
    return -1

def spearman_correlation_metric(metric_values, C_n):
    # Compute Spearman's rank correlation coefficient
    n = len(metric_values)
    ranks = {v: i for i, v in enumerate(sorted(set(metric_values)), start=1)}
    sorted_metrics = [ranks[v] for v in metric_values]
    sorted_C_n = [ranks[c] for c in C_n]

    numerator = sum((sorted_metrics[i] - sorted_C_n[i]) ** 2 for i in range(n))
    denominator = n * (n**2 - 1) / 12
    return 1 - (6 * numerator) / denominator

def gaussian_elimination(matrix):
    # Perform Gaussian elimination to compute the determinant
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i + max(range(i, rows), key=lambda r: abs(matrix[r][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(cols):
            if j != i:
                factor = Fraction(matrix[j][i], matrix[i][i])
                for k in range(cols):
                    matrix[j][k] -= factor * matrix[i][k]

    det = 1
    for i in range(rows):
        det *= matrix[i][i]
    return det

def compute_determinant(input_space):
    # Compute the determinant of the input space matrix
    n = len(input_space)
    if n == 0:
        return 1
    elif n == 1:
        return 1
    else:
        matrix = [[0] * (n + 1) for _ in range(n + 1)]
        for i, (j, val) in enumerate(input_space):
            matrix[i][j] = val
            matrix[i][-1] += val
            matrix[-1][i] += val
        return gaussian_elimination(matrix)

def compute_C_n(n):
    # Compute the deterministic protocol complexity C(n)
    return n

metric_values = []
C_n = []
instances_tested = 0
n_max = 0

```

```

for n in [5, 10, 15, 20, 30, 40]:
    if n > 40:
        break

    for _ in range(5):
        problem = generate_communication_problem(n)
        input_space = construct_input_space(problem)
        metric_value = compute_determinant(input_space)
        C_n.append(compute_C_n(n))

        instances_tested += 1
        n_max = max(n_max, n)
        metric_values.append(metric_value)

if len(metric_values) < 30:
    return {
        "metric_name": "Euler characteristic",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

correlation_coefficient = spearman_correlation_metric(metric_values, C_n)
return {
    "metric_name": "Euler characteristic",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": abs(correlation_coefficient) >= 0.95,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f46a43e2.py", line 136, in <module>
    result = run_trial(seed)
            ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f46a43e2.py", line 103, in run_trial
    metric_value = compute_determinant(input_space)
                  ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f46a43e2.py", line 85, in compute_deter
    return gaussian_elimination(matrix)
           ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f46a43e2.py", line 64, in gaussian_elim
    factor = Fraction(matrix[j][i], matrix[i][i])
            ~~~~~
File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error before producing data necessary for evaluating the conjecture. | next: Investigate and fix the crash in the test code, then rerun the test with multiple random seeds to evaluate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15499
2	propose	ollama_remote	glm4:latest	0	0	13867
3	preregistration	ollama_remote	glm4:latest	0	0	9971
4	novelty	ollama_remote	glm4:latest	0	0	8679
5	novelty	ollama_remote	glm4:latest	0	0	17749
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16382
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21088
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13426
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	79388

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	judge	ollama_remote	glm4:latest	0	0	39523

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 235573 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/e945a38a700b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e945a38a700b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e945a38a700b.tar.gz (if generated)